SALESFORCE ADMIN COURSE — EPISODE 28

Lightning Email Templates in Salesforce

Complete Guide — Modern Template Builder, Handlebars Syntax & Practical Hands-on

- What are Lightning Templates?
- Drag-and-Drop Visual Builder
- Handlebars Merge Language (HML)
- Recipient vs. Sender Context
- Building Interactive Welcome Email
- Classic (SML) vs. Lightning (HML)

EPISODE	DURATION	LEVEL	CORE UI
28 of Series	~22 Minutes	Intermediate	Lightning Experience

📋 Table of Contents

1. Introduction & Course Context — 0:00 – 0:50
2. What are Lightning Email Templates & Why Use Them? — 0:50 – 3:15
3. Classic vs. Lightning Templates (The Core Divide) — 3:15 – 5:00
4. Handlebars Merge Language (HML) & Recipient vs. Sender — 5:00 – 7:30
5. Hands-on Demo Part 1: Creating the Template & Related Entity — 7:30 – 10:00
6. Hands-on Demo Part 2: Lightning Email Template Builder — 10:00 – 15:30
7. Adding Visual Assets: Images & Call-to-Action Buttons — 15:30 – 17:00
8. Previewing & Live Record Sending from Contact — 17:00 – 20:00
9. Admin Checklist & Troubleshooting — 21:00 – 21:45
10. Summary & Key Takeaways — 21:45 – 22:30

SECTION 1

Introduction & Course Context

⏱ 0:00 – 0:50

Welcome back to the **Salesforce Admin Course!** In Episode 27, we mastered **Classic Email Templates** and learned how Salesforce Merge Language (SML) injects record data into outbound emails.

While Classic templates are functional, designing them often requires writing raw HTML code or dealing with rigid Classic Letterheads. Today, Salesforce provides a modern, intuitive, and visually stunning alternative designed natively for Lightning Experience: **Lightning Email Templates** and the visual **Email Template Builder**!

SECTION 2

What are Lightning Email Templates & Why Do We Need Them?

⏱ 0:50 – 3:15

📌 OFFICIAL DEFINITION

A Lightning Email Template is a reusable, responsive email template built directly inside Lightning Experience. It supports the modern Handlebars Merge Language (HML) and allows admins and users to assemble rich layouts using drag-and-drop components (buttons, images, dividers, rich text) without writing code.

💡 THE CORE BUSINESS PROBLEM SOLVED

Suppose your sales reps reach out to 50 new clients every morning. Typing welcome emails manually leads to inconsistent formatting, mismatched fonts, and typos. With a Lightning Email Template, reps click **Insert Template**, and Salesforce instantly produces a polished email addressed to *Rahul* or *Priya* with correct account details and clickable buttons in one second!

SECTION 3 — CRITICAL ARCHITECTURE

Classic Email Templates vs. Lightning Email Templates

⏱ 3:15 – 5:00

Criterion	📜 Classic Email Templates (Ep 27)	⚡ Lightning Email Templates (Ep 28)
Merge Language	Salesforce Merge Language (SML)	**Handlebars Merge Language (HML)**
Merge Tag Syntax	`{!Contact.FirstName}` (single curly braces)	`{{{Recipient.FirstName}}}` (triple curly braces)
Creation Experience	Setup wizard with HTML code textarea	Drag-and-drop **Email Template Builder**
Navigation Path	Setup → Classic Email Templates	**App Launcher → Email Templates**
Interactive Components	Manual HTML tables and inline CSS buttons	Out-of-the-box **Button & Image components**

🧠 EXAM RULE OF THUMB

If you see **single curly braces + exclamation**: `{!Contact.FirstName}` ➔ **Classic (SML)**.
If you see **triple curly braces**: `{{{Recipient.FirstName}}}` ➔ **Lightning (HML)**!

SECTION 4 — CORE CONCEPT

Understanding Handlebars Merge Language & Context

⏱ 5:00 – 7:30

Notice how Lightning templates use `{{{Recipient.FirstName}}}` instead of hardcoding `{{{Contact.FirstName}}}`. Lightning introduces **contextual merge roles**:

RECIPIENT CONTEXT

{{{Recipient.FieldName}}}

Represents the **person receiving the email** (whether they are a Contact, Lead, or Person Account).

- `{{{Recipient.FirstName}}}`
- `{{{Recipient.LastName}}}`
- `{{{Recipient.Email}}}`
- `{{{Recipient.Phone}}}`

SENDER CONTEXT

{{{Sender.FieldName}}}

Represents the **logged-in employee sending the email**.

- `{{{Sender.FirstName}}}`
- `{{{Sender.LastName}}}`
- `{{{Sender.Title}}}`
- `{{{Sender.Email}}}`

Example Template Structure:

```
Subject: Welcome to our Community, {{{Recipient.FirstName}}}! Hello {{{Recipient.FirstName}}}, Thank you for registering with us. We have your email recorded as {{{Recipient.Email}}}. If you have any questions, reach out to me directly. Best Regards, {{{Sender.FirstName}}} {{{Sender.LastName}}} {{{Sender.Title}}}
```

SECTIONS 5 & 6 — STEP-BY-STEP HANDS-ON

🛠️ Creating a Template & Using the Email Template Builder

⏱ 7:30 – 15:30

Part A: Setting Template Parameters

1. Open **App Launcher (9 dots)** → Search for `Email Templates` and open it.
2. Click **New Email Template**.
3. **Template Name:** `Customer Welcome Email`.
4. **Related Entity Type:** Select `Contact` (ensures Contact fields are accessible in the picker).
5. **Folder:** Choose *Public Email Templates* (or a department folder).
6. **Subject:** `Welcome, {{{Recipient.FirstName}}}!`.
7. Click **Save**.
8. On the template record page, click the **Edit in Builder** button.

Part B: Assembling Components in Email Template Builder

✍️ Rich Text Component

Main body paragraphs, headings, and HML merge fields.

🖼️ Image Component

Company logos, promotional headers, and product banners.

🔘 Button Component

Clickable call-to-actions (e.g., "Visit Our Website", "Learn More").

➖ Divider Component

Clean visual line breaks separating sections.

🧱 Row Layouts

1-column, 2-column, or asymmetrical grid columns.

🎨 Style Palette

Background colors, padding, borders, and font sizing.

SECTIONS 7 & 8 — LIVE RECORD TEST

Previewing Dynamic Values & Sending from Contact

⏱ 17:00 – 20:00

1. In the Builder header, click **Preview**.
2. Under *Preview As*, search for a sample Contact: `Rahul Sharma`.
3. Verify that `{{{Recipient.FirstName}}}` seamlessly renders as **"Rahul"** and `{{{Recipient.Email}}}` renders as **"rahul@example.com"**!
4. Toggle mobile and desktop preview modes to check responsive layout.
5. Click **Save & Back**.

Using the Template in Real Life:

1. Open any active Contact record (e.g., *Priya Sharma*).
2. In the Activity timeline on the right, click the **Email** tab.
3. Click the **Insert Template** icon (document icon at bottom).
4. Select `Customer Welcome Email`.
5. Notice how the email body automatically greets: *"Hello Priya!"*
6. Click **Send**. Next time you send to Rahul, it says *"Hello Rahul!"* with zero extra setup! 🎉

SECTION 9

Admin Best Practices Checklist

⏱ 21:00 – 21:45

- **Always Set Related Entity Type:** Setting this to *Contact*, *Lead*, or *Opportunity* ensures the merge picker offers the exact fields you need.
- **Always Use the Merge Picker:** Avoid typing curly braces manually; clicking the { } icon prevents bracket typos.
- **Preview Before Publishing:** Test with both filled-out and partially completed contact records to see how missing phone/title fields render.
- **Clear Naming Standards:** Use functional names like *"Q4 Onboarding Follow-Up"* rather than generic names like *"Template A"*.
- **Keep Syntax Clean:** Do not mix Classic `{!Contact.Name}` and Lightning `{{{Recipient.Name}}}` tags in the same template.

🎯 Key Takeaways from Episode 28

01

Lightning Email Templates are modern, responsive templates created and managed directly inside Lightning Experience.

02

Managed via **App Launcher ➔ Email Templates** rather than the Classic Setup tree.

03

Powered by **Handlebars Merge Language (HML)** utilizing triple curly braces: `{{{Recipient.FirstName}}}`.

04

Recipient tags represent the customer receiving the message; **Sender** tags represent the employee sending it.

05

The **Email Template Builder** lets admins drag and drop rich text, images, buttons, and column layouts with zero code.

06

Core Golden Rule: **Create Once ➔ Automate Personalization ➔ Send to Hundreds with 1 Click**!

COMING UP NEXT — EPISODE 29

Next Core Salesforce Admin Feature

Stay tuned as we continue building practical, job-ready Salesforce administration and automation skills!

These notes are an official companion to the YouTube video. Practice hands-on in your Salesforce Developer Org!